

EMBRY-RIDDLE AERONAUTICAL UNIVERSITY

EP 496/497

SPACE SYSTEMS DESIGN

Team mDLP

Final Design Review



Submitted to: Engineering Physics Review Board

Submitted by: Danny Charpentier, Logan Shaffer, Anuhya Suhas, Skylar Wardlaw

April 27th, 2026

Acknowledgements

The mDLP senior design team would like to thank the hard-working collaborators that ensured the success of this project. We would like to thank our senior design project coordinator, William MacKunis, PhD. We would also like to thank the student and faculty researchers of the Space and Atmospheric Instrumentation Lab: Robert Clayton, PhD, Aroh Barjatya, PhD, and Nathan Graves, PhD Candidate. We appreciate your guidance, expertise, and lab space during this endeavor. A special thanks to Bill Russo of the CNC machine shop in the College of Engineering for manufacturing assistance that aided in the assembly of product deliverables.

Abstract

The miniaturized Double Langmuir Probe (mDLP) is a plasma diagnostic instrument developed to measure plasma properties such as electron density and temperature in environments where spacecraft charging presents data acquisition challenges. Unlike traditional plasma probes, the mDLP employs a differential voltage sweeping technique, biasing each of the probes to equal but opposite voltages, thereby eliminating the need for a common stationary chassis ground. As the probes generate a dynamic, floating ground, the spacecraft charging effects are reduced. The project integrates four unique subsystems; electrical, mechanical, firmware, and software – all of which are necessary for the cohesive architecture of the mDLP suite – allowing for controlled voltage sweeps ($\pm 15\text{V}$ and $\pm 60\text{V}$ for WSLP) and for high-resolution current measurements to be collected. The architecture emphasizes reliable operation of the instrument under thermal and vibrational conditions associated with launch sequences, with key objectives of obtaining electrical isolation, thermal management, mechanical stability data acquisition reliability. All subsystems are curated in accordance with applicable NASA standards to support flight-ready implementations for mission success. Further experimental validation was conducted throughout the course of the academic semester, with the mDLP suite successfully generating observable current-voltage (I-V) traces. Versions of instrument code for several different applications of the instrument were developed and tested. These results confirm the instruments' ability to capture plasma parameters from the electron characteristics in plasma environments. Additionally, software enhancements demonstrate reliable data acquisition, handling, storage, and visualization with payback capabilities for both testing and in-flight operations. The resulting mDLP suite provides a robust and efficient solution for in-situ plasma diagnostics and for the future spaceflight missions requiring these project deliverables for mission success.

Table of Contents

Acknowledgements	2
Abstract	3
Table of Figures	6
Glossary	9
1. Introduction	10
1.1 Overview of the mDLP	10
1.2 Plasma Environments and the Ionosphere	10
1.3 Goal of the Instrument	12
2. Establishing Needs	13
2.1 General Needs	13
2.2 Specific Needs	13
2.3 Alternative Solutions	16
2.4 Goals and Objectives	17
2.5 Measures of Effectiveness (MOE)	19
3. System Concept Development and Problem Definition	21
4. Solution: Senior Design	22
4.1 Design Process Overview	22
4.2 Subsystem Overview	22
4.2.1 Electrical	22
4.2.2 Mechanical	25
4.2.3 Firmware	32
4.2.4 Software	38
5. Results and Evaluation	47
5.1 Direct Outcomes	47
5.1.1 Electrical Results	47
5.1.2 Mechanical Results	50
5.1.3 Firmware Results	62
5.1.4 Software Results	67
5. Discussions	71
5.2.1 Electrical Discussions	71

<i>5.2.2 Mechanical Discussions</i>	73
<i>5.2.3 Firmware Discussions</i>	74
<i>5.2.4 Software Discussions</i>	76
<i>6. Conclusion</i>	77
<i>References</i>	78
<i>Appendix A- Software Architecture</i>	81
<i>Appendix B- Mechanical Architecture</i>	87

Table of Figures

Fig. 1: Electron density per cubic centimeter at different altitudes within the ionosphere.	11
Fig. 2: An example current-voltage (IV) trace depicting different regions of interest for plasma characteristics.....	11
Fig. 3: The current mDLP instrument and power board.....	14
Fig. 4: Prototypes of the SAIL designed planar probes for CubeSat applications.	16
Fig. 5: Previous heated WSLP prototype.....	17
Fig. 6: Final CubeSat PCB board statistics.....	23
Fig. 7: Final dimensions of the CubeSat PCB.	24
Fig. 8: Multilayer stack breakdown within Ultiboard.....	24
Fig. 9: CAD Assembly of the EP 496 Prototype heated WSLP.	26
Fig. 10: SolidWorks generated Bill of Materials (BOM) for the assembly of the heated WSLP.	27
Fig. 11: Component breakdown of the heated WSLP.	28
Fig. 12: Final print of the High-temperature resin mechanical guards.	29
Fig. 13: WSLP configuration within the Plasma Chamber for testing.	30
Fig. 14: Boom attachment for the integration of the WSLP onto the sounding rocket boom.	31
Fig. 15: Cartoon of the functionality of the heating element within the WSLP design.....	31
Fig. 16: Final SolidWorks mock-up for the full sounding rocket boom assembly with the WSLP.	32
Fig. 17: The initial variable sweep pattern.....	34
Fig. 18: The slower 5-second sweeps generated by the 115.2kB rate.	34
Fig. 19: The proposed half-sweep pattern for F04.....	35
Fig. 20: The first attempt of buffer logic for F03.	36
Fig. 21: Software System Overview Block Diagram.....	40
Fig. 22: Software System Architecture (Model View Controller).....	41
Fig. 23: Software Live Streaming Data Flow Block Diagram (Serial).....	43
Fig. 24: Software Playback Data Flow Block Diagram (File).....	45
Fig. 25: Software Recording Data Flow Block Diagram.....	46
Fig. 26: Software Parser Finite State Machine Block Diagram.....	46
Fig. 27: Side-by-side comparison of the shapes of the initial mDLP instrument vs. the CubeSat instrument.	48
Fig. 28: Example of one reconfiguration of the power system schematic to optimize space available on the CubeSat footprint.....	49
Fig. 29: Photo of the final design schematic of the CubeSat PCB, highlighting the integration between the power system and instrumentation modules.....	50
Fig. 30: Setup and Results for the bench testing of the prototype WSLP.	52
Fig. 31: Setup of the bench testing for the WSLP prototype.	52
Fig. 32: DC power supply setup (left) and the resulting temperature via thermocouple measurement in degrees Celsius.	53
Fig. 33: Thermocouple testing of the heating probe to measure WSLP surface temperature.	53

Fig. 34: Plasma chamber set up for the WSLP.	54
Fig. 35: Voltage sweep diagnostics from the aluminum SAIL WSLP (left) and the EP 497 WSLP prototype (right).	54
Fig. 36: SolidWorks iteration of modeling internal threads within the mechanical guard design.	58
Fig. 37: Filament iterations of the mechanical guard (with internal threads) with the proposed crush washer aside it for insertion.	58
Fig. 38: Resin printing iterations of the mechanical guard of various degrees of failure.	59
Fig. 39: Final resin print results of the mechanical guard.	59
Fig. 40: Filament printed boom attachment on a 1" diameter boom.	60
Fig. 41: Final manufactured assembly of the heated WSLP on a 1-meter boom for sounding rocket applications.	61
Fig. 42: Final configuration of the WSLP within the boom attachment.	61
Fig. 43: The setup for plasma chamber testing.	62
Fig. 44: The raw current and voltage graphs (left) and differential current and voltage graphs (right) in lower temperature and pressure plasma (top) and higher temperature and pressure plasma (bottom).	63
Fig. 45: The hardware setup and GUI integration results.	64
Fig. 46: The bench testing results for each stage of sounding rocket code implementation, chronological from left to right as follows: transmission, DAC setpoint computation, and measurement.	65
Fig. 47: The initial results from satellite board integration.	66
Fig. 48: The differential UART bench testing setup.	66
Fig. 49: A video still showing the satellite code sweeping on command.	67
Fig. 50: Screenshot of the Graphical User Interface Program.	68
Fig. 51: Comparison table of Expected Behavior and Actual Output.	69
Fig. 52: Screenshot of the UI panels in the GUI.	70
Figure A 1: Software System Overview Block Diagram.	81
Figure A 2: Software System Architecture (Model View Controller).	81
Figure A 3: Software Live Streaming Data Flow Block Diagram (Serial).	82
Figure A 4: Software Playback Data Flow Block Diagram (File).	83
Figure A 5: Software Recording Data Flow Block Diagram.	84
Figure A 6: Software Parser Finite State Machine Block Diagram.	84
Figure A 7: Comparison table of Expected Behavior and Actual Output.	84
Figure A 8: Screenshot of the Graphical User Interface Program.	85
Figure A 9: Screenshot of the UI panels in the GUI.	86
Figure B 1: Alumina Insulator CAD Model.	87
Figure B 2: Cartridge Heater CAD Model.	87

Figure B 3: Mechanical Guard CAD Model.....	88
Figure B 4: WSLP Threaded Sheath CAD Model.....	88
Figure B 5: Heated WSLP Final CAD Assembly	89
Figure B 6: Clamshell half Boom Attachment CAD Model.....	89
Figure B 7: Boom Attachment CAD Model.....	90
Figure B 8: Final boom assembly of the WSLP	91
Figure B 9: Insulator CAD Drawing.....	92
Figure B 10: Cartridge Heater CAD Drawing	92
Figure B 11: Heater probe assembly CAD Drawing	93
Figure B 12: Threaded Probe Sheath CAD Drawing.....	93
Figure B 13: Mechanical guard CAD Drawing	94
Figure B 14: High temperature resin data sheet.....	94
Figure B 15: Theoretical control model for the thermal subsystem	95

Glossary

ADC: Analog-to-Digital Converter

CAD: Computer Aided Design

COM: Communications

CPU: Central Processing Unit

CSV: Comma-separated values

DAC: Digital-to-Analog Converter

DMA: Direct Memory Access

FPS: Frames Per Second

FRAM: Ferroelectric Random Access Memory

FSM: Finite State Machine

GUI: Graphical User Interface

HF: High Frequency

IDE: Integrated Development Environment

IPC: Inter-Process Communication

mDLP: Miniaturized Double Langmuir Probe

MVC: Model-View-Controller

OS: Operating System

PCB: Printed Circuit Board

RAM: Random Access Memory

UART: Universal Asynchronous Receiver/Transmitter

USB: Universal Serial Bus

WSLP: Wide-Sweeping Langmuir Probe

1. Introduction

1.1 Overview of the mDLP

The miniaturized Double Langmuir Probe (mDLP) is a diagnostic instrument designed to measure electron density and temperature in a plasma environment. Unlike traditional single Langmuir probes, which rely on a reference to a common spacecraft ground, the mDLP operates using a differential sweeping voltage bias mechanism between two probes. In this configuration, the probes are driven with equal but opposite voltages, and the measured current is taken between them. As a result, the system does not require a reference to the spacecraft's electrical ground, as it generates its own dynamic ground point between the two probe's voltage biases. Thus, the mDLP is capable of eliminating issues that arise with spacecraft charging in plasma environments, which is the primary purpose of the project [1].

Spacecraft charging occurs when an instrument is supplied with a voltage and begins to attract different plasma species within the environment towards the probe [2]. If the size ratio of the instrument area to the spacecraft area is smaller than a ratio 1:1000, any shared electrical ground between the two generates a non-negligible charging effect of the entire spacecraft. Additionally, the proximity of the probe sensors to the spacecraft also increases charging effects. The solution to mitigating this is the unique orientation of the Langmuir Probe configuration in the mDLP. Having two separate, opposing probes allows for opposite biasing of the voltages supplied to the probes. This voltage differential forces an isolated ground to be generated, where the collection of electron-induced currents from the external plasma does not interact with the internal spacecraft ground [1] [2].

The traditional Langmuir probes are better suited for missions like sounding rockets, where the probes are relatively much smaller by surface area than the spacecraft itself, thus reducing charging effects on the spacecraft; for CubeSat missions, however, this is not the case. As the probes are mounted upon a smaller spacecraft, the ability to reduce charging becomes a more localized and severe problem. With this in mind, this project aims to investigate mDLP variations for both sounding rocket and satellite mission specifications, in order to create flight-ready instruments.

1.2 Plasma Environments and the Ionosphere

The ionosphere is the dynamic region of the upper atmosphere covering altitudes from ~60 – 600km. Its defining characteristic is the presence of free electrons and ions produced through the ionizing effect due primarily to the solar ultraviolet radiation these altitudes experience [3]. These electrons and ions, while free-floating, are close enough to each other to generate what is known as a quasi-neutral plasma environment. As such, the ionosphere experiences a variety of daytime-nighttime changes within the vast range of altitudes the plasma environment covers. Thus, the ideal region for instrumentation probes to gather consistent data is within the upper most altitudes—the F Region, with the largest amount of electron presences per cubic centimeter—in order to observe the highly ionized species [2] [3] [4]. Fig. 1 demonstrates the electron density variability within daytime-nighttime cycles at different altitudes within the ionosphere.

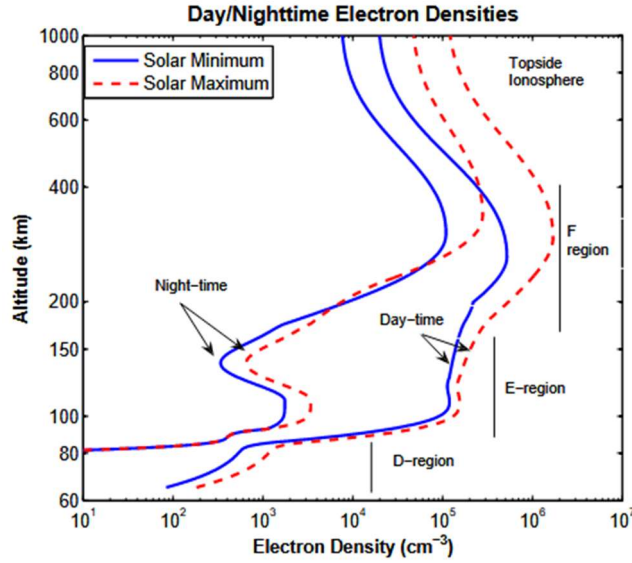


Fig. 1: Electron density per cubic centimeter at different altitudes within the ionosphere.

However, the observation of ion densities has been pioneered for decades. The more novel approach to investigating the plasma environment is to produce readings of thermal electron temperatures. Within this topmost layer of the ionosphere, vast amounts of photoionization generate an increase in energetic photoelectrons, and as these particles collide with the plasma environment, electrons lose energy in the form of heat. This allows for the opportunity to derive the temperature of electrons [1] [3] [5].

This average kinetic energy correlates with the current collection seen within the exponential defining the electron retardation region of a standard current-voltage (I-V) curve. I-V traces display a range of plasma parameters as a voltage is swept across a probe. Fig. 2 displays a generic I-V curve covering all the major regions of useful information to later be used in characteristic derivations.

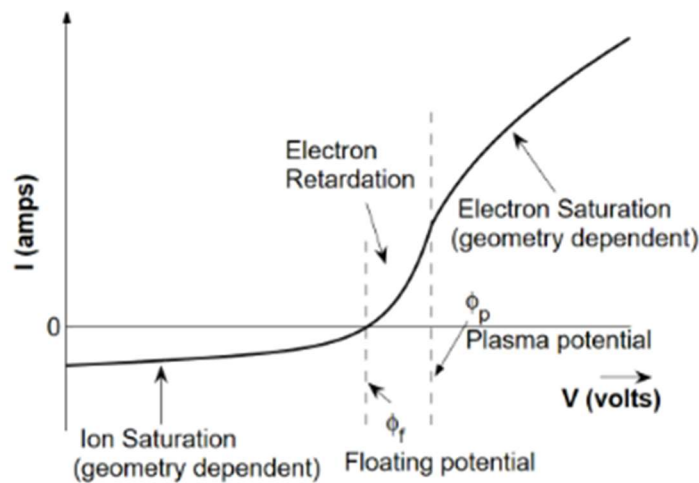


Fig. 2: An example current-voltage (IV) trace depicting different regions of interest for plasma characteristics.

The major parameters the mDLP cares for include the following: floating and plasma potentials (ϕ_f and ϕ_p), electron densities (n_e), and the electron retardation regions of measured current ($I_e(\phi)$). With these given parameters, a relationship can be defined between electron thermal current and the average kinetic energy of the electrons, T_e , as seen in Equation (1) [4] [6].

$$I_e(\phi) = I_{the} \exp\left(\frac{e(\phi - \phi_p)}{k_B T_e}\right) \quad (1)$$

Relating the net electron retardation region as a function of the potential to the potential relative to the plasma potential, ϕ_p , the Boltzmann constant, k_B , electron temperature, T_e , and the fundamental electron charge, e . To round out the full expression, the thermal current measured is displayed in Equation (2); however, this is dependent upon geometry of the probe, and in the mDLP proposal, probes are both planar and cylindrical, which would involve both the area and a geometry-dependent factor, β ($0 \leq \beta \leq 0.5$). Notably, this parameter would only be utilized within the electron saturation region, not the retardation region.

$$I_{thermal,e} = \frac{1}{2} n_e e A \sqrt{\frac{k_B T_e}{2\pi m_e}} \quad (2)$$

This equation has dependencies on the species parameters, which are electrons in this case. From these equations, the electron temperature can be found and analyzed for further plasma characterization.

1.3 Goal of the Instrument

The mDLP is engineered as a plasma and atmospheric characterization instrument. Due to its extremely sensitive measuring characteristics—electron density, temperature, and other plasma parameters are derivable. The Earth's ionosphere is a region of the atmosphere composed of plasma, which is fundamental to the numerous networks and communication devices that interact with it. These networks include GPS signals, satellite linking and communications, high frequency (HF) radio transmission, and radar system navigation [2] [3] [5].

With a more comprehensive understanding of the parameters that define plasma in the ionospheric region, the scientific community will be able to more accurately model atmospheric drag on spacecrafts, predict space weather conditions and anomaly responses, and design more resilient aerospace systems [3]. Atmospheric instruments such as the mDLP allow for the development of a more accurate model of the Earth's ionospheric plasma and pave the way for advancements in modern technological infrastructure and scientific research [1].

2. Establishing Needs

2.1 General Needs

The mDLP is a novel instrument relying on plasma and electronics theory, designed for various space-related missions. The instrumentation board has been fully constructed in association with the Space and Atmospheric Instrumentation Lab (SAIL), thus the desire for this senior design team to generate flight-ready variations of the instrument for both sounding rocket and satellite missions.

2.2 Specific Needs

Plasma Chamber Diagnostics:

The original mDLP instrument was designed as a proof-of-concept for this novel method of electron-based plasma characterization and a spacecraft floating ground. The preliminary testing was performed in a plasma chamber to cross validate results with experimentally verified plasma diagnostic instruments. While much of the hardware for this already existed prior to the senior design team's involvement, some modifications to the firmware and software were required.

The firmware subsystem is the mDLP instrument's embedded software. While a baseline version of the instrument code did previously exist and perform basic functionality, some modifications were necessary to improve data collection in a diagnostic application. The original sweep pattern was variable, which was decided to be unnecessary for future testing in the chamber; instead, a new constant sweep pattern was implemented in the firmware. Additionally, integration with a live GUI for real-time graphical data viewer was desired for plasma chamber usage. More about the GUI will be presented in the software section. For verification, the firmware was tested in the chamber as a diagnostic of the plasma characteristics observed.

The need for a program to display real-time data from the instrument is necessary to ensure that the correct data is getting transmitted while on its mission. This program needs to extend beyond displaying basic real-time data because there is no software that exists commercial-off-the-shelf for this specific purpose. The program will function as a graphical user interface (GUI) and additionally function as a recording and playback device for data captured. It is also necessary to store the data in a CSV format to then later playback or scientifically analyze by exporting the file to be manipulated in post processing.

Sounding Rocket Mission:

The mDLP, initially designed for application in a plasma chamber to validate the experimental performance of this novel instrument design in a plasma environment, had to be modified to better suit a sounding rocket application. The primary goal for this team is to generate a flight-ready sounding rocket version of the instrument to fly on NASA's Resolute Sounding Rocket as part of the HELiophysics Strategic Technology Office (HESTO) Pathfinder Project in 2027 [1]. A summary of the different subsystem modifications required for this version of the mDLP is as follows.

displaying one data type, status panel unfinished, no console, and no testing and integration with other systems just to name a few issues.

The duration of this academic semester was spent refactoring, refining, and developing off these problems. The following table outlines the key milestones for the “build-ready” product for the GUI. The objectives of the EP 497 timeline for the software are as follows: frontend, backend, and validation of the software. These objectives are in preparation for extensive use with the serial connection with the instrument for the 2026 – 2027 year of data collection, validation, and mission. Several tradeoffs come with development using Python. The advantage of using Python is the extensive number of libraries available with documentation and online guides, it is high level, so it is easier to interpret and develop in, it has automatic memory management so no need to worry about garbage collection. The largest disadvantages are its slow computing speed compared to Java, C++, or Julia, large memory consumption, and threading limitations [26]. Python was still chosen despite these disadvantages because they can be worked around and kept in mind when developing.

Frontend: Plots - There is to be minimum of 6 plots for DAC Voltage, Integrator Voltage, ADC A Current, ADC B Current, Differential Voltage, and Differential Current all vs time with ability to change axis, scaling, and auto recenter	S01
Frontend: UI Panels – There shall be a control panel in the main window allows user to switch between streaming over serial and playback from file, clear any data, reset program, and display console (debugging and configuration purposes)	S02
Frontend: Serial connection panel – functionality for user to select BAUD rate, connect / disconnect, discover ports	S02a
Frontend: Playback panel – functionality for user to select file, start / stop playback, playback rate, and bulk plot option	S02b
Frontend: Recording panel – functionality for user to select data folder file will be saved, start / stop recording	S02c
Frontend: Status panel – visual for user to see mode, recording or not, system status, and data points currently plotted / recorded	S02d
Backend: Data Storage – The GUI shall have a robust, efficient data storage system with performance first able to be read from by subsystems for plotting and saving	S03
Backend: Parser – There shall be a parser that takes input in the form of binary protocol and converts into data structure for data storage subsystem	S04
Backend: Serial Acquisition – The GUI shall connect to the instrument over Serial at specified BAUD Rate and COM port, the data stream is sent to parser to be processed	S05
Backend: Recording – The GUI shall save all data being input (serial or file) to a CSV file from a toggle in the control panel	S06
Backend: Playback – The GUI shall play back any data input from the recorded CSV file as if coming from serial input.	S07

Validation: Testing – Perform operation with serial connected and recording for 15min and 20min. Additionally, run system for extended time period to test for memory leaks, crashes, other issues	S08
Validation: Firmware Integration – Incorporate output of firmware in form on data packets (binary protocol) to input serial subsystem and iterate through issues between systems	I02

Table: The software subsystem semester milestones with the associated callout code.

The system is separated into two main sections, communication inside the program, and outside the program. This is best represented by the system overview block diagram as seen below:

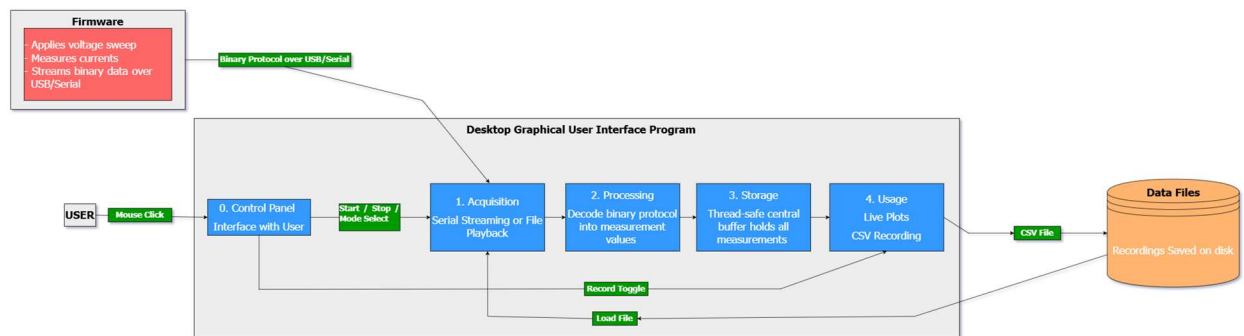


Fig. 21: Software System Overview Block Diagram.

The grey blocks represent separate components of each subsystem, where the blue blocks are plain text explanations, pseudocode, or script names with short descriptions. The green blocks represent data, signals, and other items transferring between larger blocks. The orange cylinder represents a database but for intents and purposes it represents the disk on the computer. All other block colors and shapes are described when necessary and will follow the same rules for the remainder of the software block diagrams.

The system overview block diagram shows the data flow between the user, firmware, data files and the GUI. The flow occurs when the user clicks anything on the control panel (i.e. Start Serial Streaming) in addition to the firmware system constantly outputting data when connected over serial. That data flows into the acquisition block, then gets sent to the processing block which handles the parsing from binary protocol to measurement values, then it gets sent to the storage block which contains a thread safe buffer to hold all the measurements, to which the last block can access it to plot the data and save it to a CSV file in the disk to be accessed later.

The design pattern used to create the GUI is the model, view, controller architecture. It can best be described with the diagram as seen below:

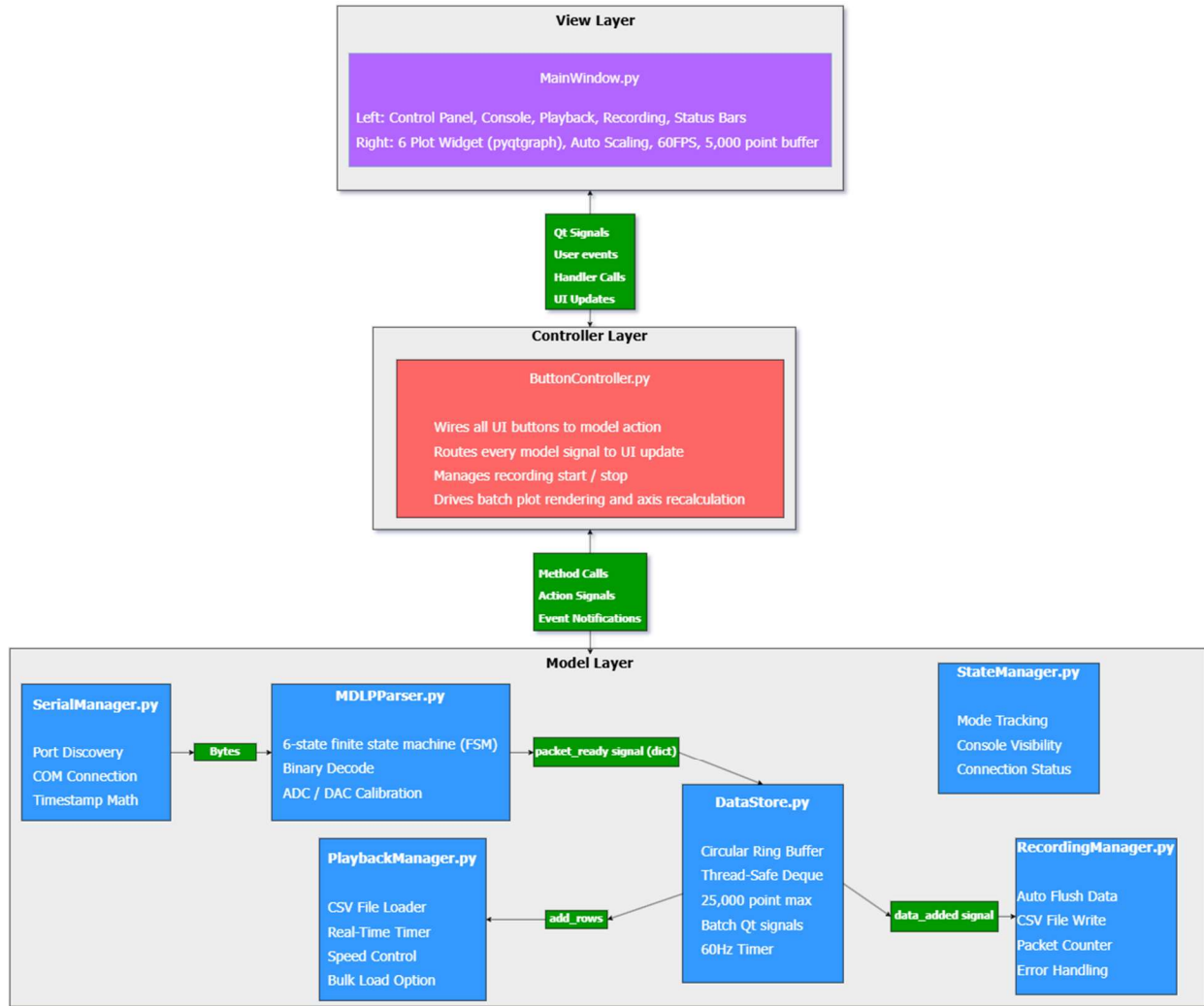


Fig. 22: Software System Architecture (Model View Controller).

This diagram shows there are three separate blocks with titles: Model, View, and Controller layers. The model components handle all data management and business logic, these are completely independent of the user interface. The view is responsible for the presentation, focused solely on display. The controller mediates between the view and model layers [29].

The model components within the diagram above show the following scripts:

- `StateManager`: manages the application state and transitions.
- `DataStore`: Central data buffer with thread safe operations and batch processing. Able to hold up to 25,000 points at a time.
- `SerialManager`: Handles serial communication with the instrument
- `PlaybackManager`: Manages recorded data playback functionality
- `RecordingManager`: Handles recording the data to a CSV file
- `MDLPParser`: 6 State FSM that handles decoding binary into measurement values

The view component contains the script:

- **MainWindow:** Solely UI implementation using PyQt6 widgets, layouts, and other visuals
 - Uses PyQtGraph for real-time data visualization
 - Implements the PlotRingBuffer for plot data storage

The controller component contains the script:

- **ButtonController:** Handles all user interactions (button clicks, UI events, etc.)

The benefits of using this implementation of MVC are:

- **Testability:** All layers can be tested separately without affecting core logic of another
- **Flexibility:** UI can be modified without affecting the functionality behind events
- **Reusability:** Model components can work with different or changing UIs
- **Maintainability:** Clear separation enables for easier debugging and updates

The architecture is implemented fully in this program as the scripts demonstrate a distinct separation of concerns between layers, objects created separately and follows the goal of "loose coupling with high cohesion" that Freeman et al. emphasize as essential for maintainable object-oriented design [29].

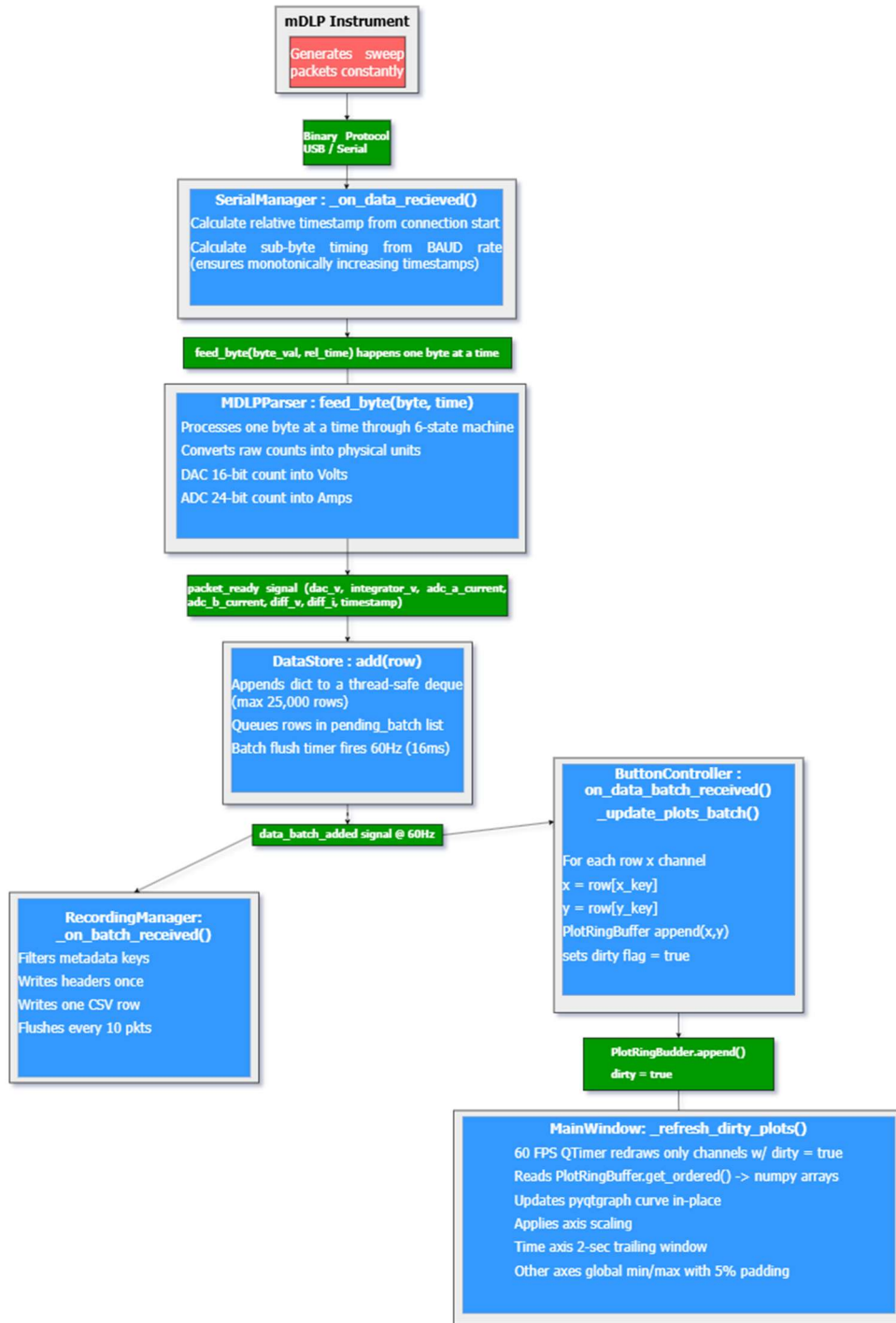


Fig. 23: Software Live Streaming Data Flow Block Diagram (Serial).

This block diagram shows the data flow for the live streamed data connected over serial. It follows this flow: Data Generation & Acquisition, Data Processing, Data Storage & Distribution, and Real-Time Visualization. Each of these components are necessary for the live streaming to work and are detailed below. First in the data generation & acquisition section, the physical instrument continuously generates sweep packets containing the measurement data, it is transmitted in binary protocol via USB/Serial connection. The SerialManager acts as the first layer in the data pipeline where it calculates the relative timestamp from connection start, calculates the sub-byte timing from the BAUD rate to ensure monotonically increasing timestamps, feeds the individual bytes to the parser, and runs constantly on a background thread to maintain UI responsiveness. Then in the data processing pipeline, the MDLPParser contains a state machine for converting the raw digital counts to real world units such as Volts and Amps. It assembles the completed data packets and emits a signal once completed structured dictionary containing: `dac_v`, `integrator_v`, `adc_a_current`, `adc_b_current`, `packet_id`, and `timestamp`. Then the ButtonController uses a for each loop to extract the timestamp and measurement value for each row and channel. It then calls `PlotRingBuffer.append(x,y)` and sets `dirty = true` which marks plots needing refreshing with the data just sent. Finally in the real-time visualization, MainWindow uses a 60FPS QTimer, which redraws only channels with `dirty = true`, reading from the ring buffer and updates the plots using PyQtGraph. It also has auto scaling for the Y axis, a 2 second trailing window for the X axis and global 5% padding for all axes.

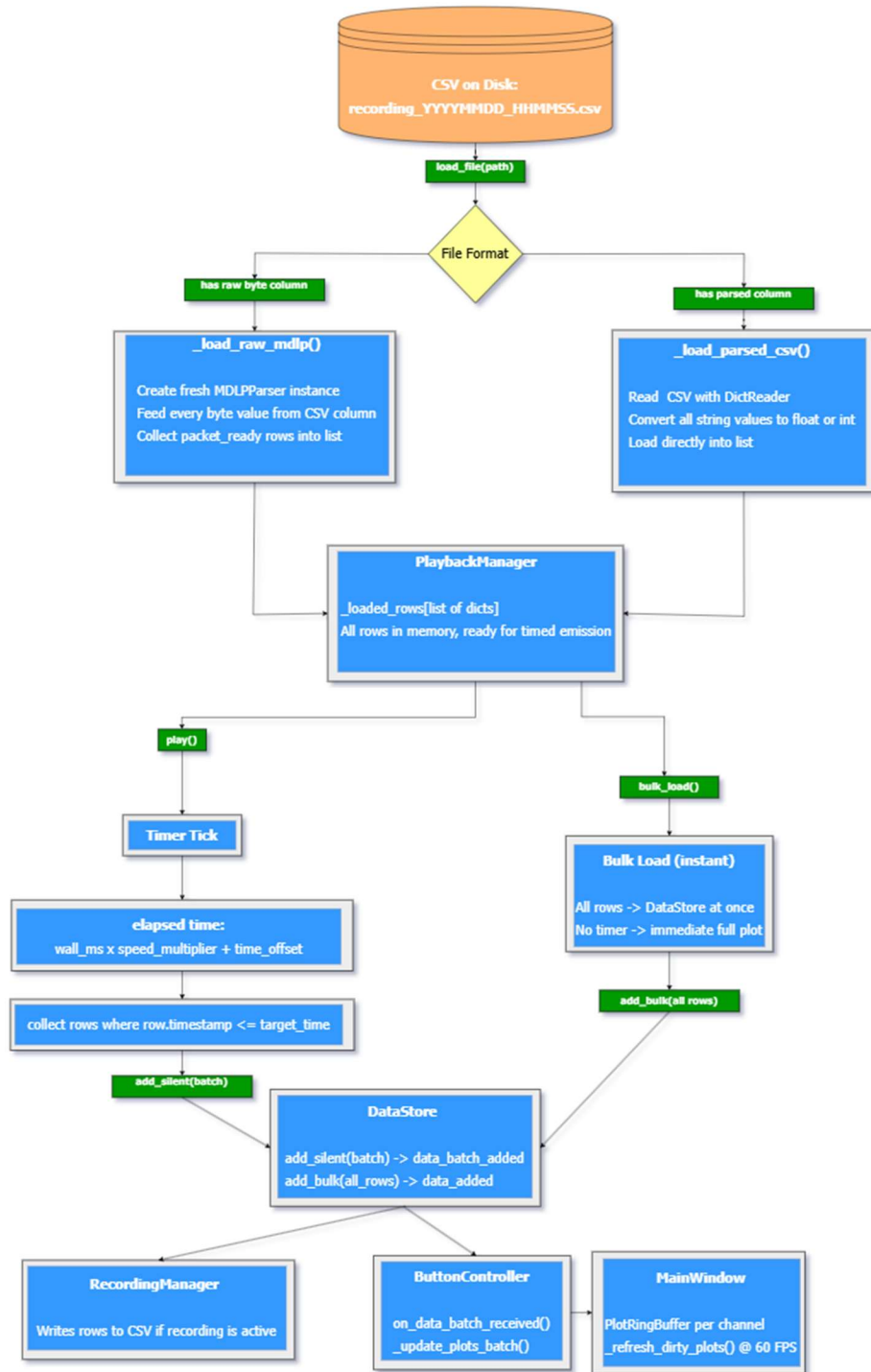


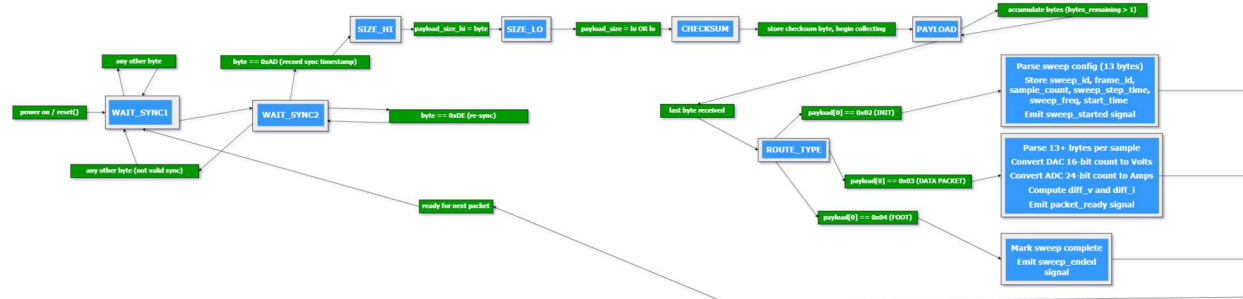
Fig. 24: Software Playback Data Flow Block Diagram (File).

This block diagram shows the data flow for the data played back from a CSV file. It follows this flow: File Selection & Loading, PlaybackManager Data Preparation, Playback Modes, Data Distribution Pipeline, and Output Processing. First, recorded data files stored on the disk are loaded into the GUI from user selection, the files either contain the raw byte data or parsed

columns. The system determines the loading approach depending on the CSV structure. If it contains a raw byte column path, then a fresh parser is created and goes through byte by byte processing then sent to the PlaybackManager. If the file instead contained parsed columns, then it is directly fed to the PlaybackManager. Once the data reaches the PlaybackManager it goes through the same process as explained earlier into storage. If the user selects regular playback (not bulk), then there is an adjustable timer that processes data as if it was live, this happens with bath emission. If the user selects bulk plotting, then one large batch is instantly plotted containing the whole dataset. The data goes through the same output processing as earlier described through the ButtonController and MainWindow.



This block diagram shows the data flow recording data to a CSV file. It follows this flow: Recording Initialization & Setup, Recording Pipeline Activation, Data Processing & Filtering, Continuous Data Writing, and File Management & Output. Firstly, there must be data inside DataStore to be able to start recording as well as an input from the user in the form of clicking the Start button in the UI. Once both occur, the Boolean of 'is recording' is set to true and the recording pipeline activates. Here, the system creates a file named recording_YYYYMMDD_HHMMSS.csv in the ISO 8601 format. It then opens the CSV and prepares internal buffers for metadata filtering. The metadata that is removed contains irrelevant keys such as debug info and timestamps so that only measurement relevant data is kept. File headers in the form of column names are created and filled. To write the data in rows, the script processes each dictionary in the data batch and iterates row by row. The buffer is consistently flushed every 10 packets to balance performance and mitigating data loss. Finally, once the user presses stop recording or the program closes, the recording file is closed safely and can be read using any desktop application that views CSVs such as Microsoft Excel or be played back into the GUI.



This block diagram shows the finite state machine design in the parser. It follows this flow: Raw Byte Input & Synchronization, Packet Type Classification, Data Packet Processing Pipeline, and Packet Assembly & Emission. At the start, the parser receives individual bytes one at a time from SerialManager, they arrive in a continuous stream from the instrument. This stream contains sync bytes, headers, payload, and possible noise. At WAIT_SYNC1 state, it waits for the first sync byte (0xDE), discarding all non-sync bytes, then advances to WAIT_SYNC2. Here, it waits for the second sync byte (0xAD) which confirms the packet started, if not the correct byte, then it returns to the first state. The parser identifies between three different packet types, being TYPE_INIT, TYPE_DATA_PACKET, TYPE_FOOT. The TYPE_INIT packet contains the setup configuration data and establishes the start of the instrument's data packet. The TYPE_DATA_PACKET contains the measurement data values in counts. The TYPE_FOOT indicates the completion of the sweep and signals to end the current cycle. For TYPE_DATA_PACKET, the parser combines sequential bytes into complete values while handling the proper endianness. It does this by converting a 16-bit input to DAC voltage and a 24-bit ADC to current measurements. Upon completion of the packet, the data is assembled into a dictionary data structure containing key value pairs and adds timing information from SerialManager and relevant metadata such as packet ID, frame count, and statuses.

The reason a state machine was used instead of parsing the data using traditional methods such as standard regular expressions (Regex) despite it using an internal state machine, it does not have the ability to track separations between packets. Therefore, if a Regex was used instead, it would cause an error to occur if any packets were lost or separated into two packets whereas this implementation can continue as it only has finite states it can travel to with continuous recovery built in (returning to state 1 automatically). Additional benefits to this version is that it has low latency because it processes bytes immediately, it is memory efficient as no buffer is required, it operates continuously, and requires low computational power.

The methodology used to verify the performance criteria of the software system is done mainly by inspection as all that is needed to verify the data flow is a visual examination of the plots, CSV file, correct interaction in the UI. Other methods used to verify included testing and analysis with specific code segments. This method was particularly useful when validating the timing of the parsing logic and performance of the program under load.

5. Results and Evaluation

5.1 Direct Outcomes

The following subsections give a detailed discussion on the direct outcomes regarding each subsystem's tasks and the associated deliverables outlined in the previous section.

5.1.1 Electrical Results

The main goal of the electrical subsystem was the redesign the previous mDLP circuit board and power board into a more compact and CubeSat-compatible format, while also retaining all of the

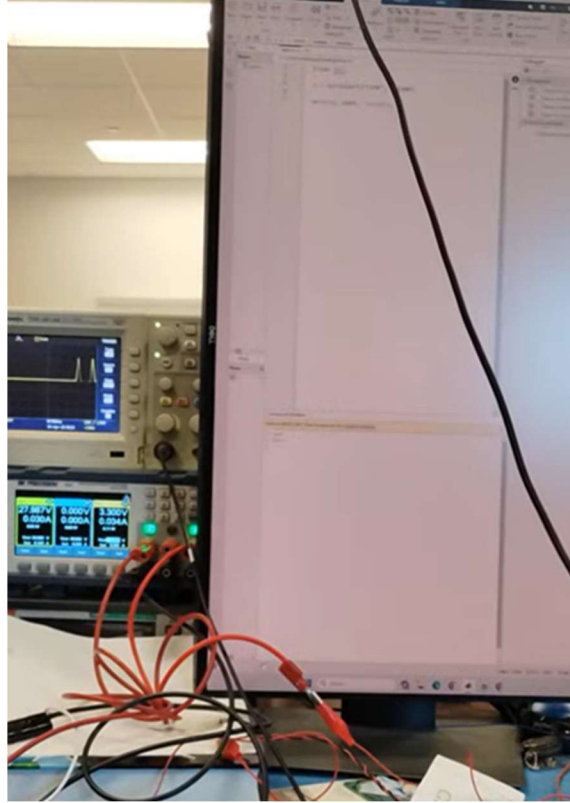


Fig. 49: A video still showing the satellite code sweeping on command.

As presented in this section, all versions of the mDLP firmware operate as desired and have been tested and integrated with the other subsystems. This marks a successful development of the firmware subsystem in this project.

5.1.4 Software Results

The primary objective of the software system was to develop a fully functional GUI that can acquire, display, record, and playback data from the mDLP instrument in real time. The completed system is shown in the result below and meets all the primary requirements:

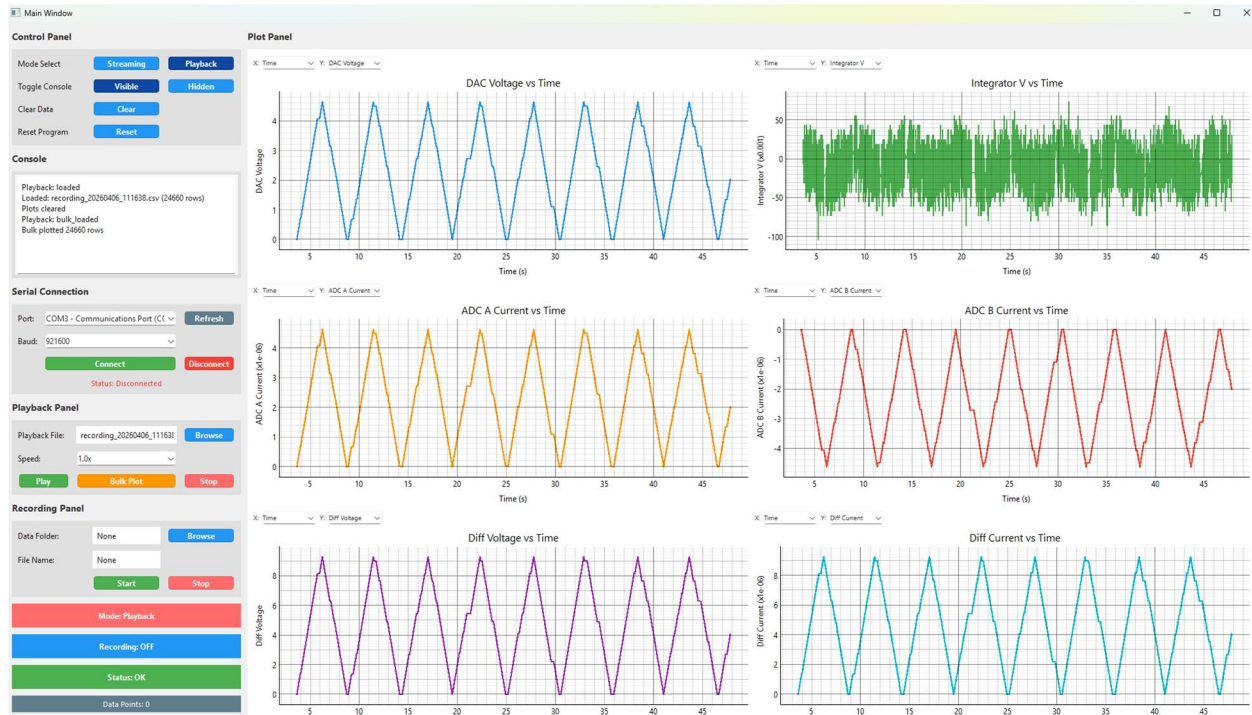


Fig. 50: Screenshot of the Graphical User Interface Program.

The following table shows the tangible deliverables status according to the requirement and ID:

Requirement	ID	Status
Frontend: Plots - There is to be minimum of 6 plots for DAC Voltage, Integrator Voltage, ADC A Current, ADC B Current, Differential Voltage, and Differential Current all vs time with ability to change axis, scaling, and auto recenter	S01	Met
Frontend: UI Panels – There shall be a control panel in the main window allows user to switch between streaming over serial and playback from file, clear any data, reset program, and display console (debugging and configuration purposes)	S02	Met
Frontend: Serial connection panel – functionality for user to select BAUD rate, connect / disconnect, discover ports	S02a	Met
Frontend: Playback panel – functionality for user to select file, start / stop playback, playback rate, and bulk plot option	S02b	Met
Frontend: Recording panel – functionality for user to select data folder file will be saved, start / stop recording	S02c	Met
Frontend: Status panel – visual for user to see mode, recording or not, system status, and data points currently plotted / recorded	S02d	Met
Backend: Data Storage – The GUI shall have a robust, efficient data storage system with performance first able to be read from by subsystems for plotting and saving	S03	Met
Backend: Parser – There shall be a parser that takes input in the form of binary protocol and converts into data structure for data storage subsystem	S04	Met

Backend: Serial Acquisition – The GUI shall connect to the instrument over Serial at specified BAUD Rate and COM port, the data stream is sent to parser to be processed	S05	Met
Backend: Recording – The GUI shall save all data being input (serial or file) to a CSV file from a toggle in the control panel	S06	Met
Backend: Playback – The GUI shall play back any data input from the recorded CSV file as if coming from serial input.	S07	Met
Validation: Testing – Perform operation with serial connected and recording for 15min and 20min. Additionally, run system for extended time period to test for memory leaks, crashes, other issues	S08	Met
Validation: Firmware Integration – Incorporate output of firmware in form on data packets (binary protocol) to input serial subsystem and iterate through issues between systems	I02	Partially

Table: Software requirement and status completion table

As it can be seen from the above table, all requirements that were proposed have been successfully and completely met. The following will describe how these were achieved.

S01 – Plots:

All six channels are rendered simultaneously with the correct waveform shapes as verified by the table below:

Channel	Expected Behavior	Actual Output	Result
DAC Voltage	Linear sweep ramp (sawtooth)	Clean triangular waveform, 0–4.5 V	Confirmed
Integrator V	High-frequency noise signal	Broadband noise, –100 to +50 mV	Confirmed
ADC A Current	Proportional to DAC sweep	Sawtooth waveform, 0–2.5 A	Confirmed
ADC B Current	Inverted probe response	Inverted sawtooth, 0 to –4 A	Confirmed
Diff Voltage	Differential of DAC channels	Triangle waveform, 0–9 V	Confirmed
Diff Current	Differential of ADC channels	Triangle waveform, 0–8 A	Confirmed

Fig. 51: Comparison table of Expected Behavior and Actual Output.

Each plot features independently changeable axes depending on the dropdown selected by the user, auto scaling and recentering abilities, thus fully satisfying the requirements.

S02 / S02a / S02b / S02c / S02d — UI Panels:

All panels are present and functional in the GUI. The COM port and BAUD rate selection panel exists with a refresh button and connect / disconnect buttons. The file browser, speed selector, play, stop, and plot controls exist inside the playback panel. The folder and file name fields with browse, start, stop controls exist inside the recording panel. The status labels featuring mode (playback / streaming), recording on / off, status OK banner, and data point counter are featured at the bottom of the panels. A zoomed in screenshot of the UI panels is shown below:

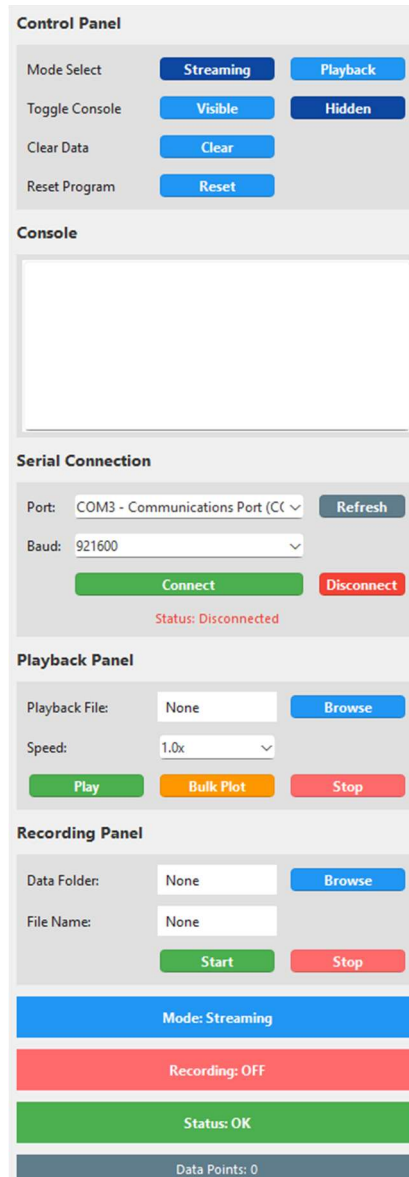


Fig. 52: Screenshot of the UI panels in the GUI.

S03 – Data Storage:

The data storage implements a thread-safe deque (double-ended queue) with max size of 25,000 points. This means that it allows multiple threads to add or remove items from both ends concurrently without causing data corruption. It uses batch emission at 60Hz to keep fast data collection separate from the systems trying to use that data (plots / saving). The system has been confirmed to handle over 690,000 rows of parsed data at a single time.

S04 – Parser:

The parser processes binary protocol using a 6-state machine. It converts the 16 bit DAC counts to volts and 24-bit ADC counts to amps. It emits a structured dictionary via a signal directly to DataStore. All functions of the parser work efficiently and correctly.

S05 – Data Acquisition:

The SerialManager discovers any available COM ports vis pyserial. It allows the BAUD rate to be configured and COM port to be selected from the UI panel. The script runs on a background thread using QThread to ensure any operations don't interfere with the UI. The system has been used on multiple machines and confirmed to recognize all connections to the instrument.

S06 - Recording:

The RecordingManager writes to CSV rows in real time with timestamped filenames according to ISO 8601. The headers are written once on the first packet and the file is flushed every 10 packets for data security. The start and stop controls are directly from the control panel. The output of CSV has been validated within Microsoft Excel.

S07 – Playback:

The PlaybackManager plays the CSV data through the same DataStore pipeline as the live serial input. The use of timed playback (timer tick) to emulate the live input of the instrument data as well as bulk loading into the storage work properly as confirmed through the console and plots.

S08 – Testing:

The GUI has been confirmed to be able to record up to 20min of data input from the instrument without errors, hanging, or crashing. All the data was saved to a CSV file in length of over 690,000 rows of parsed data. This file was then played back using the Playback panel both in the timer and bulk loading. It has also worked running at 10x regular speed during playback. Memory profiling and CPU usage were not officially documented during testing, however, no issues were reported in the console or hanging, so it can be considered a success.

I02 – Integration:

Firmware integration was achieved to the extent that binary protocol parsed from the live output during bench testing. Live testing directly connected to the plasma chamber remains in the outstanding milestone.

5. Discussions

5.2.1 Electrical Discussions

Project

Success:

Based on the defined electrical requirements, the electrical subsystem is to be considered a strong success. The final mDLP electronics design successfully generated the required +/- 15V differential sweep necessary for the Langmuir probe's critical plasma diagnostics while also

5.2.4 Software Discussions

Project Success:

Against the defined requirements, the software section is considered a great success. All frontend requirements (S01 – S02d) are fully implemented and validated visually. All backend requirements (S03 – S07) are operational and demonstrated real recorded data from the instrument and are validated through testing, including S08. The delivered system satisfies 12 out of the 13 requirements completely, with the last requirement partially fulfilled. The clean patterns across all the channels confirm that the binary protocol parser correctly decodes and converts the instrument sweep packets to physical units. The MVC architecture cleanly separates the concerns across the Model, View, and Controller scripts to support long-term maintainability.

Limitations:

The requirement of firmware integration (I02) was limited to bench testing in the SAIL laboratory and on recordings. The live instrument integration within the plasma chamber remains to be tested. The program was only tested on two machines, being a laptop and desktop. Both of these machines had windows 11 OS and similar screen sizes and resolution, therefore no tests were done to confirm it would work on other OS and machines such as Linux or Mac.

Areas of Improvement:

Converting the backend from Python to a language such as Julia would prove extremely useful in processing high volumes of data, especially when plotting in Python. This would directly improve any limitations with using a higher BAUD rate. It would also improve the ability to store upwards of 25,000 points at once and speed up plotting in both bulk and regular playback. In addition to this, implementing a form of automatic profiling such as memory or CPU usage would be streamlined. A separate configuration file filled with settings such as the max size for the buffer can be used so other machines can utilize different storage options.

Lessons Learned:

Designing a program around a professional framework like the MVC architecture takes careful consideration into every element that the program is supposed to accomplish. Using signals from PyQt6's signal-slot mechanism was crucial to prevent coupling between the layers. This made unit testing and adjustments much easier during development. Developing real-time systems is unique to experiences such as development for simulations. This became evidently clear when working through issues in timing, buffer overflow, memory allocation, unit conversions, data preservation, and much more. Furthermore, utilizing tools such as Git for version control was essential, as tracking any changes became instantly clear to an outside developer or for reverting changes to a prior version. Moreover, documentation and reality often differ, which became evident when developing the parser as the actual byte ordering was different than what was expected. Overall, the lessons learned prove that real-time scientific system development requires attention to performance, data integrity, hardware integration, and user challenges that were not evidently clear during the initial design phases.

6. Conclusion

The mDLP, to reiterate, is a diagnostic tool utilized for the extraction of electron information within plasma environments. As such, the goal of this specific instrument configuration allows for a key functionality interference to be minimized—spacecraft charging. With the opposing voltages supplied to either probe, this differential in generated currents creates a “floating” ground, thus protecting the spacecraft chassis from experiencing any external charging effects from the usage of electrically conductive probes.

Under the guidance of the research team within SAIL, the senior design team was tasked with the unique challenge of designing, implementing, and configuring the mDLP for various spacecraft missions for plasma diagnostics. The following subsystems were defined: electrical, mechanical, firmware, and software.

The electrical subsystem showed successful reconfiguration of the previously constructed mDLP power and instrument board to conform to the standards and specifications for that of a CubeSat PCB. Due to the strict regulations of the CubeSat formfactor, the power and instrument board required merging, thus the integration of both power supply regulation and board configuration took priority for the senior design team.

The mechanical subsystem was tasked with the challenge of redesigning the novel heated WSLP. As such, the results display a successful manufacturing and testing of the flight-ready prototype for submission to the third-party sounding rocket program that proposed the mDLP integration for scientific measurements within the Earth’s upper ionosphere. Furthermore, the full assembly of the WSLP shows all mechanical structures, material selection, and thermal loadings were deliberated in excruciating detail in order to ensure the success of any flight requiring the extraction of the electron characteristics within plasma environments.

The firmware subsystem focused on the development of three versions of the instrument code to best suit a plasma chamber diagnostic, sounding rocket, and satellite application of the instrument. The design tasks for each version of the code were different and required novel implementations. The code was tested in the plasma chamber and in a bench testing format, and was integrated with the corresponding hardware and software subsystems. While further development and testing at SAIL is possible, all firmware design specifications desired for this senior design project were successfully completed.

The software subsystem set out to design, develop, and test a graphical user interface capable of displaying real-time science data from the instrument. It additionally sought to have functionality to record and play trials back from user defined input. The resulting software displays accurate and complete information from the instrument as well as being able to record and playback data from the trials. All requirements were fulfilled and therefore the subsystem was a great success.

References

- [1] R. Clayton, "Development and Testing of a miniaturized Double Langmuir Probe for Small-Satellite Platforms," Nasa Proposal ID. 22-HTIDS22-0018, 2022.
- [2] NASA, "Chapter 2: Introduction to Physics of Charging and Discharging," in *SciTechBook*, pp. 7-30.
- [3] D. Gallagher, "The Earth's Plasmasphere," NASA, 26 April 2023. [Online]. Available: <https://plasmasphere.nasa.gov/>.
- [4] A. Barjatya, "Langmuir Probe Measurements in the Ionosphere," 2007.
- [5] UCAR, "The Ionosphere," UCAR: Center for Science Education, 2014. [Online]. Available: <https://scied.ucar.edu/learning-zone/atmosphere/ionosphere>.
- [6] J. Rowan, "Characterization of a Magetically Contained Hot Filament Plasma Source with a Wide-Sweeping Langmuir Probe," Embry-Riddle Aeronautical University.
- [7] NASA, "Requirements for Threaded Fastening Systems in Spaceflight Hardware," Washington, DC, 2021.
- [8] NASA, "Standard Materials and Process Requirements for Spacecraft," Washington, DC, 2023.
- [9] NASA, "Structural Design and Test Factors of Safety for Spaceflight Hardware," Washington, DC, 2022.
- [10] NASA, "Workmanship Standard for Crimping, Interconnecting Cables, Harnesses, and Wiring," Washington, DC, 2016.
- [11] NASA, "General Environmental Verification Standards (GEVS) for GSFC Flight Programs and Projects," Goddard Space Flight Center, Greenbelt, 2021.
- [12] World Material, "AISI 304 Stainless Steel Properties: Tensile Yield Strength & Hardness," [Online]. Available: <https://www.theworldmaterial.com/type-304-grade-stainless-steel/>.
- [13] P. Ceramics, "Alumina: Properties and Industrial Applications," [Online]. Available: <https://precision-ceramics.com/materials/alumina/>.

- [14] McMaster-Carr, "Nonporous Alumina Ceramic Tubes," [Online]. Available: <https://www.mcmaster.com/products/alumina/ceramic-3~/nonporous-alumina-ceramic-tubes-7/od~0-187/>.
- [15] Accuratus, "Aluminum Oxide Al₂O₃ Ceramic Properties".
- [16] 5. U. Alloys, "Exploring Thermal Conductivity in Stainless Steel".
- [17] E. Edge, "Specific Heat Capacity of Metals Table Chart".
- [18] High Temp Metals, "Inconel 600 Tech Data," [Online]. Available: <https://www.hightempmetals.com/techdata/hitempInconel600data.php>.
- [19] A. S. C. LLP, "Density of Stainless Steel 304," 2023.
- [20] H. T. Metals, "Inconel 600 Technical Data".
- [21] S. Metals, "Inconel Alloy 600".
- [22] T. E. ToolBox, "Thermal Conductivity of Metals and Alloys: Data Table & Reference Guide".
- [23] Watlow Electric Manufacturing Company, "1/8-inch FIREROD Cartridge Heater," 2016.
- [24] formlabs, "High Temp Resin".
- [25] Python Software Foundation, "Python 3.12.3 documentation," Python.org, 2024. [Online]. Available: <https://docs.python.org/3/>
- [26] L. Campagnola and PyQtGraph Contributors, "pyqtgraph 0.13.7 documentation," Read the Docs, 2024. [Online]. Available: <https://pyqtgraph.readthedocs.io/en/latest/>
- [27] C. Liechti, "pySerial's documentation," Read the Docs, 2020. [Online]. Available: <https://pyserial.readthedocs.io/en/latest/>
- [28] Riverbank Computing, "PyQt5 Reference Guide," Riverbank Computing, 2024. [Online]. Available: <https://www.riverbankcomputing.com/static/Docs/PyQt5/>
- [29] E. Freeman, E. Robson, B. Bates, and K. Sierra, *Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [30] ZeroMQ Development Team, "ZeroMQ: An open-source universal messaging library," Zeromq.org, 2024. [Online]. Available: <https://zeromq.org>
- [31] NumPy Developers, "NumPy v1.26 Manual," Numpy.org, 2024. [Online]. Available: <https://numpy.org/doc/stable/>

- [32] GeeksforGeeks, "Circular Queue in Python," GeeksforGeeks, 2023. [Online]. Available: <https://www.geeksforgeeks.org/dsa/circular-queue-in-python/>
- [33] Squareboat, "Advantages and Disadvantages of Python," Squareboat Blog, 2024. [Online]. Available: <https://www.squareboat.com/blog/advantages-and-disadvantages-of-python>.

Appendix A- Software Architecture

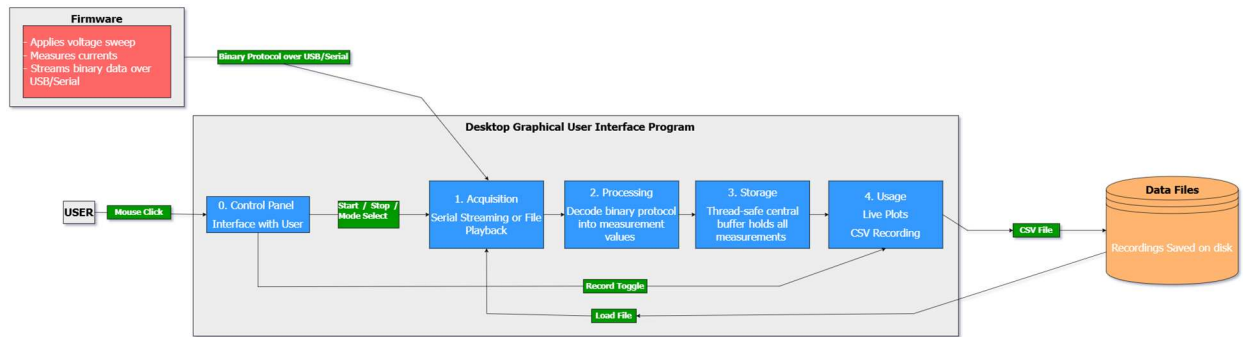


Figure A 1: Software System Overview Block Diagram

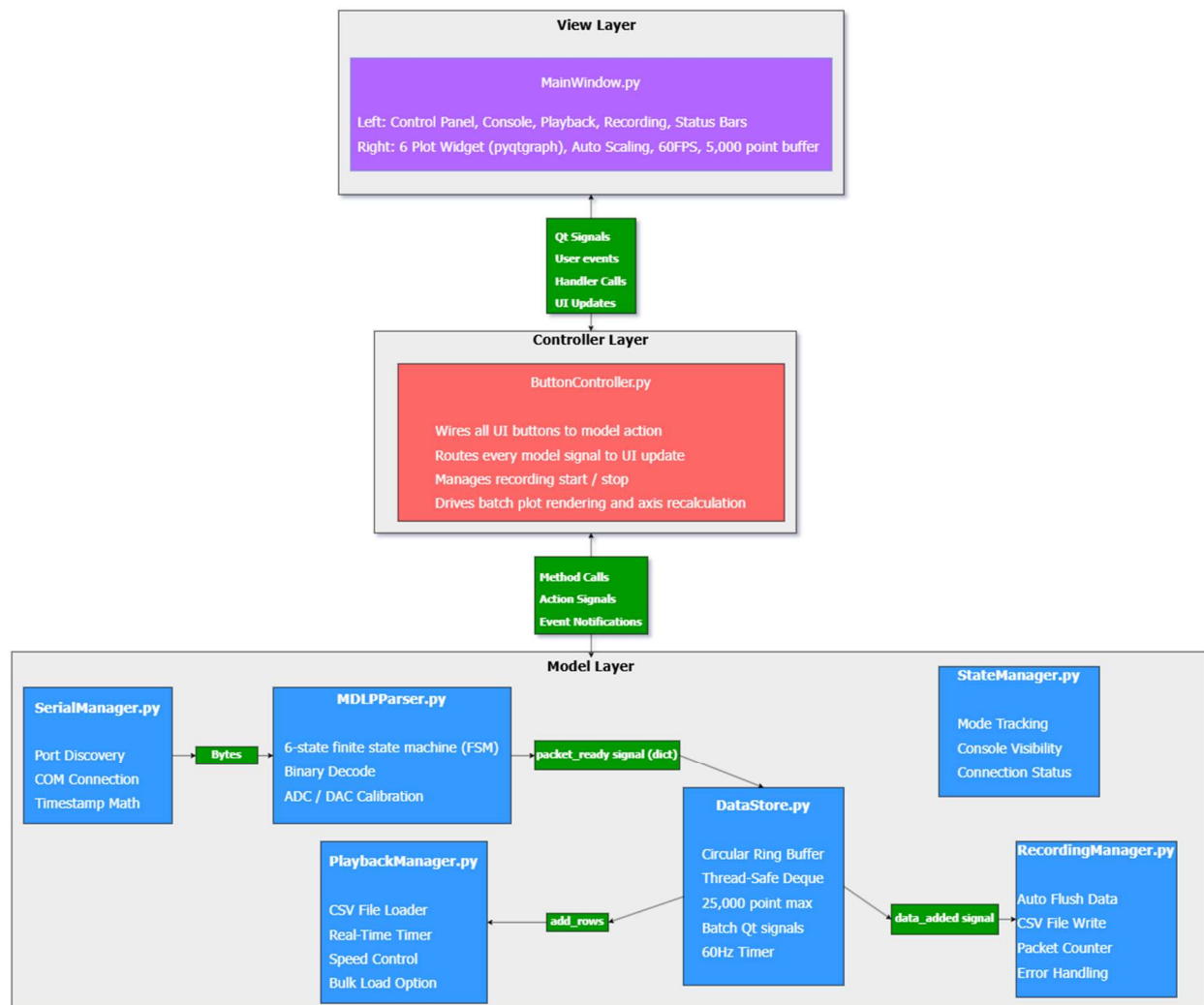


Figure A 2: Software System Architecture (Model View Controller)

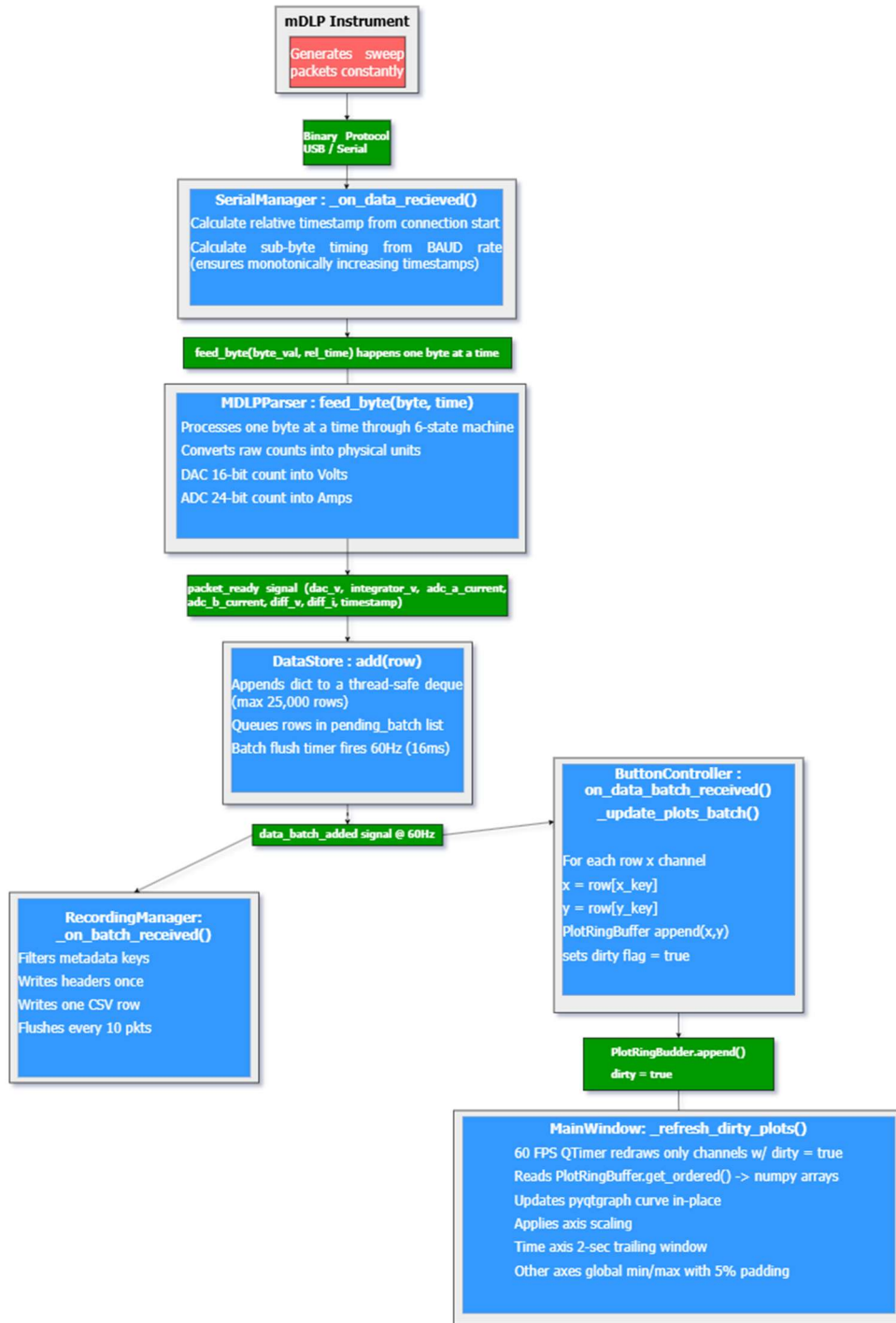


Figure A 3: Software Live Streaming Data Flow Block Diagram (Serial)

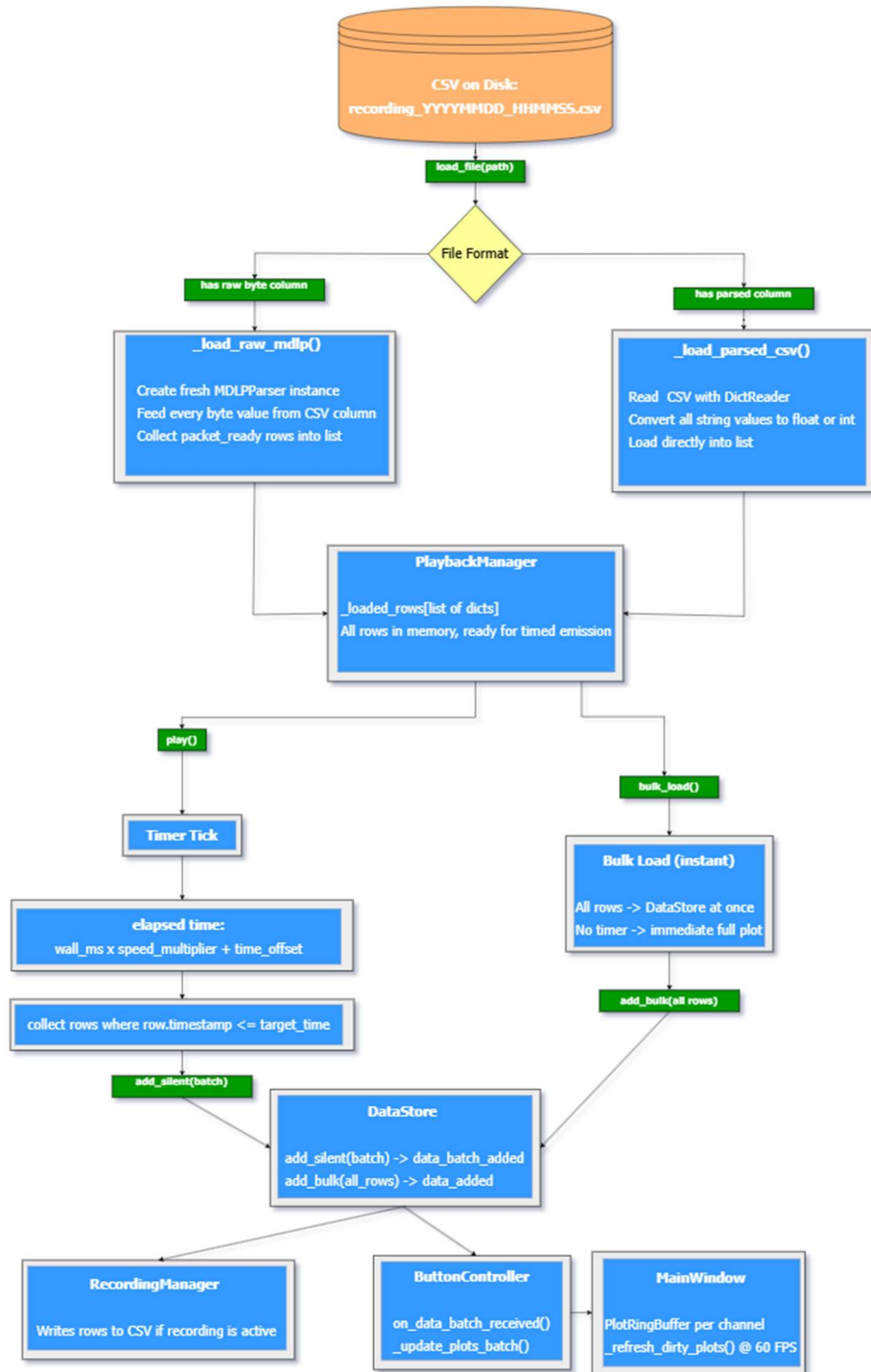


Figure A 4: Software Playback Data Flow Block Diagram (File)

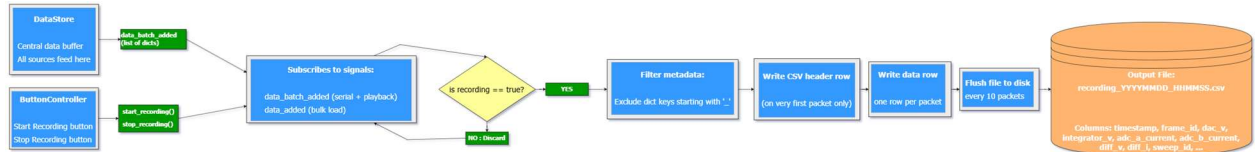


Figure A 5: Software Recording Data Flow Block Diagram

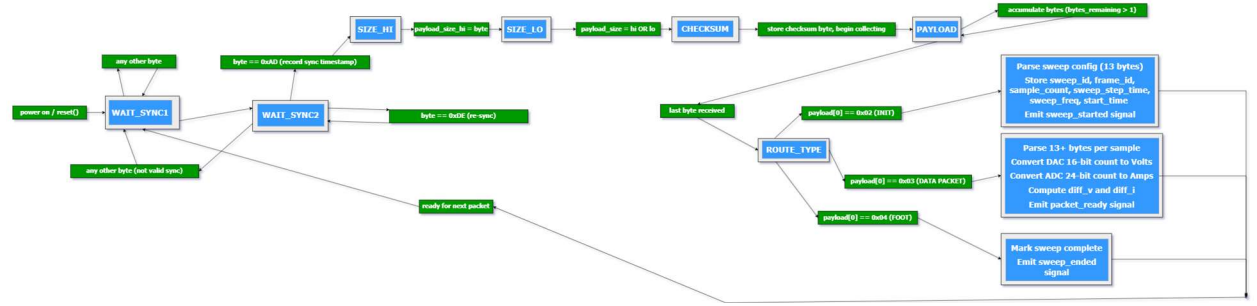


Figure A 6: Software Parser Finite State Machine Block Diagram

Channel	Expected Behavior	Actual Output	Result
DAC Voltage	Linear sweep ramp (sawtooth)	Clean triangular waveform, 0–4.5 V	Confirmed
Integrator V	High-frequency noise signal	Broadband noise, –100 to +50 mV	Confirmed
ADC A Current	Proportional to DAC sweep	Sawtooth waveform, 0–2.5 A	Confirmed
ADC B Current	Inverted probe response	Inverted sawtooth, 0 to –4 A	Confirmed
Diff Voltage	Differential of DAC channels	Triangle waveform, 0–9 V	Confirmed
Diff Current	Differential of ADC channels	Triangle waveform, 0–8 A	Confirmed

Figure A 7: Comparison table of Expected Behavior and Actual Output

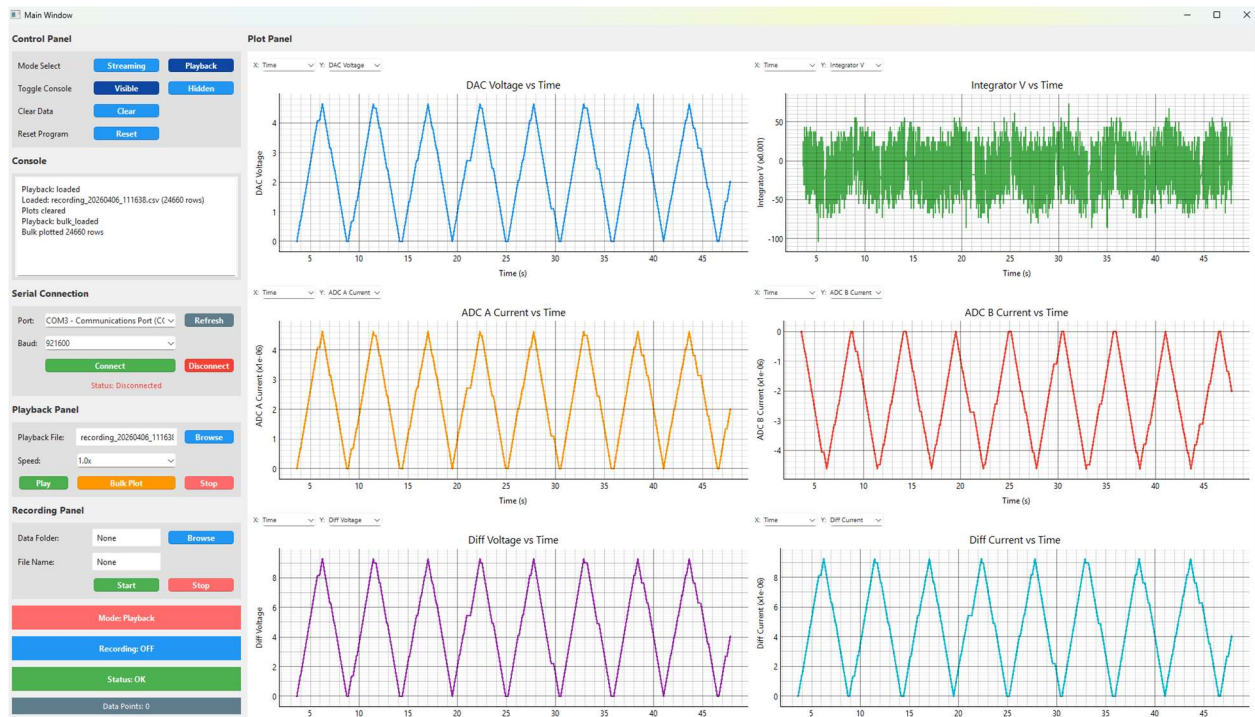


Figure A 8: Screenshot of the Graphical User Interface Program

Control Panel

Mode Select

Streaming

Playback

Toggle Console

Visible

Hidden

Clear Data

Clear

Reset Program

Reset

Console

Serial Connection

Port:

COM3 - Communications Port (C

Refresh

Baud:

921600

Connect

Disconnect

Status: Disconnected

Playback Panel

Playback File:

None

Browse

Speed:

1.0x

Play

Bulk Plot

Stop

Recording Panel

Data Folder:

None

Browse

File Name:

None

Start

Stop

Mode: Streaming

Recording: OFF

Status: OK

Data Points: 0

Figure A 9: Screenshot of the UI panels in the GUI